

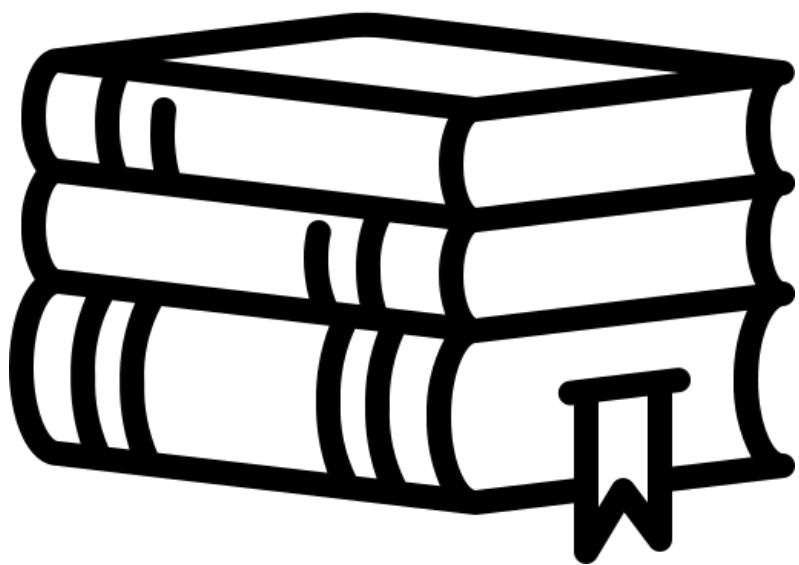
**Белорусско-Российский университет**  
Кафедра «Программное обеспечение  
информационных технологий»

# **ЭВМ, периферийные устройства и контроллеры**

## **Тема: Взаимодействие программного обеспечения с аппаратной частью**

**Кутузов** Виктор Владимирович  
Республика Беларусь, Могилев, 2025





# Рекомендуемая литература

# Рекомендуемая литература



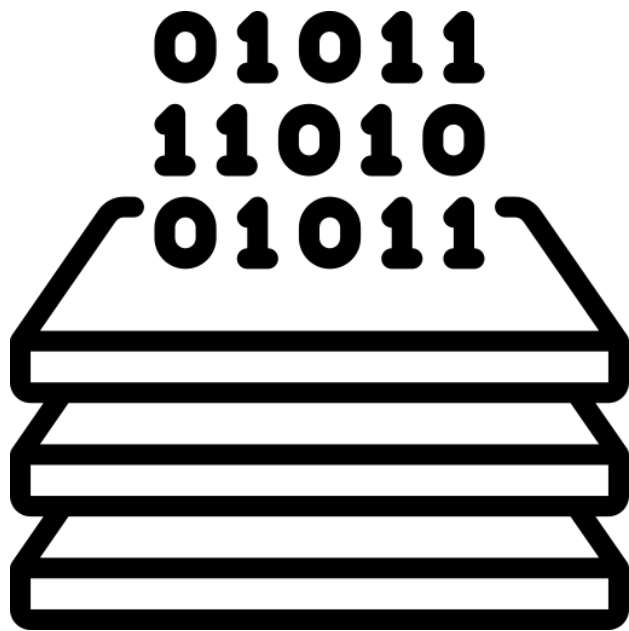
- **Таненбаум Э., Остин Т. Архитектура компьютера.** 6-е изд. – СПб.: Питер, 2013. – 816 с.  
<https://djvu.online/file/98S7ar1OrCHi5>



- **Рэндал Э. Брайант, Дэвид Р. О'Халларон Компьютерные системы: архитектура и программирование.** 3-е изд. / пер. с англ. А. Н. Киселева. – М.: ДМК Пресс, 2022. – 994 с.  
[https://t.me/Geek\\_book\\_hub/1289](https://t.me/Geek_book_hub/1289)



- **Гагарина Л. Г., Кононова А. И. Архитектура вычислительных систем и Ассемблер** с приложением методических указаний к лабораторным работам. Учебное пособие. – М.: СОЛОН-Пресс, 2019. – 368 с.  
<https://www.twirpx.com/file/2781352/>



**Взаимодействие  
программного  
обеспечения с  
аппаратной  
частью**

# Взаимодействие ПО с аппаратной частью

- **Взаимодействие программного обеспечения (ПО) с аппаратной частью** — это многослойный процесс, где высокоуровневые команды преобразуются в физические сигналы, управляющие транзисторами. Этот процесс лежит в основе работы любого компьютера.
- **Основные принципы взаимодействия:**
  - **Абстракция.** Каждый уровень скрывает детали реализации от вышележащего (например, приложение не знает, как именно SSD читает данные).
  - **Интерфейсы.** Чётко определённые точки взаимодействия: API операционных систем, системные вызовы, порты ввода-вывода.
  - **Обратная совместимость.** Аппаратная часть часто поддерживает устаревшие интерфейсы (например, эмуляция BIOS в UEFI).
  - **Иерархия привилегий.** Не все уровни имеют равный доступ к оборудованию (режимы ядра vs. пользовательский режим).

# Уровни взаимодействия

## Уровни взаимодействия: как команда доходит до транзисторов

- **1.** Высокоуровневое ПО → Прикладные программы
  - **2.** Прикладные программы → Операционная система
  - **3.** ОС → Система команд (ISA)
  - **4.** Система команд → Микроархитектура
  - **5.** Микроархитектура → Аппаратное обеспечение
- 
- **Взаимодействие ПО и аппаратной части** — это иерархия преобразований, где каждый уровень добавляет свою абстракцию, но сохраняет обратную совместимость.
  - **Современные технологии** (виртуализация, GPU-ускорение, безопасные анклавы) продолжают менять правила **взаимодействия**, но базовые принципы — системные вызовы, прерывания, DMA — остаются неизменными с 1970-х годов.
  - **Понимание этих механизмов критично для:**
    - Оптимизации производительности,
    - Отладки сложных багов,
    - Разработки драйверов и низкоуровневого ПО.
  - Чтобы глубже погрузиться в тему, попробуйте написать простой драйвер или проанализировать системные вызовы через strace в Linux — это превратит теорию в практические навыки.

# Пример

- **Пример в общем виде сквозного взаимодействия: открытие веб-страницы**
- **Сценарий:** В браузере страница запрашивает `https://example.com`.
- **Уровень 5 (ПО):**
  - JavaScript вызывает `fetch("https://example.com")`.
- **Уровень 4 (Приложение):**
  - Браузер преобразует запрос в HTTP через библиотеку (например, libcurl).
- **Уровень 3 (ОС):**
  - Системный вызов `socket()` создаёт сокет,
  - `connect()` устанавливает TCP-соединение.
- **Уровень 2 (ISA):**
  - Драйвер сети использует инструкции IN/OUT для работы с сетевым контроллером.
- **Уровень 1 (Микроархитектура):**
  - CPU обрабатывает пакеты через DMA,
  - Шифрование TLS ускоряется AES-NI.
- **Уровень 0 (Аппаратура):**
  - Сетевой чип формирует электрические сигналы для Ethernet-кабеля.



# Уровни взаимодействия ПО и аппаратуры

# Общая схема уровней абстракции

| Уровень | Название                     | Кто «видит» этот уровень? | Абстрагирование от нижележащего |
|---------|------------------------------|---------------------------|---------------------------------|
| 5       | Языки высокого уровня        | Программисты              | Синтаксис и логика программы    |
| 4       | Прикладные программы         | Пользователи              | Логика приложения               |
| 3       | Операционная система         | Системные программисты    | Аппаратные детали               |
| 2       | Система команд (ISA)         | Компиляторы, ассемблеры   | Микроархитектура                |
| 1       | Машинно-зависимое исполнение | Производители процессоров | Аппаратная реализация           |
| 0       | Аппаратное обеспечение       | Инженеры-конструкторы     | Физические законы               |

# Уровень 5: Языки высокого уровня

- **Что это:** Языки программирования, близкие к человеческому (Python, Java, C#).
- **Примеры:**
  - `print("Hello, World!")` в Python,
  - `System.out.println()` в Java.
- **Как работает:**
  - Программист пишет код, не думая об аппаратных деталях. Компилятор или интерпретатор преобразует его в более низкоуровневые инструкции (например, в байт-код Java или машинный код).
- **Ключевая идея:**
  - **Абстракция логики** — разработчик фокусируется на решении задачи, а не на том, как процессор выполнит операцию.
- **Связь с нижними уровнями:**
  - Код уровня 5 транслируется в уровень 4 (приложения) и далее через компилятор/интерпретатор «спускается» к уровню 2 (ISA).
- **Интересный факт:** Виртуальные машины (например, JVM) добавляют дополнительный уровень абстракции между языком высокого уровня и ОС, что обеспечивает кроссплатформенность.

# Уровень 4: Прикладные программы

- **Что это:** Готовые приложения, используемые конечными пользователями (браузеры, редакторы, игры и т.д.).
- **Примеры:**
  - Google Chrome,
  - Microsoft Word,
  - Игры на Unity.
- **Как работает:**
  - Приложения взаимодействуют с ОС (уровень 3) через API (например, WinAPI для Windows или POSIX для Linux), чтобы запросить ресурсы (память, файлы, сеть).
- **Ключевая идея:**
- **Абстракция функциональности** — программа не знает, как именно ОС выделит память или прочитает файл, она лишь вызывает стандартные функции.
- **Связь с нижними уровнями:**
  - Приложение «видит» только интерфейс ОС (уровень 3), но не затрагивает аппаратные детали напрямую.
  - **Важно:** Утечки памяти или ошибки в приложении (например, buffer overflow) могут нарушить работу ОС, что демонстрирует проникновение абстракций.

# Уровень 3: Операционная система

- **Что это:** Ядро ОС, драйверы, файловая система, менеджер памяти.
- **Примеры:**
  - Ядро Linux (kernel.org),
  - Драйвер видеокарты NVIDIA,
  - Файловая система NTFS.
- **Как работает:**
- ОС управляет ресурсами компьютера:
  - Распределяет память между процессами,
  - Обрабатывает прерывания от устройств,
  - Предоставляет виртуальную память.
- **Ключевая идея:**
- **Абстракция аппаратуры** — приложения работают с виртуальными ресурсами (например, «файл» вместо секторов на диске).
- **Связь с нижними уровнями:**
- ОС напрямую взаимодействует с системой команд (уровень 2), используя привилегированные инструкции (например, `syscall` в x86).
- **Пример:** Когда вы сохраняете файл в Word, ОС преобразует запрос в низкоуровневые операции:
- **Приложение → API ОС → Драйвер диска → ISA → Микроархитектура → Аппаратура.**

# Уровень 2: Система команд (ISA — Instruction Set Architecture)

- **Что это:** Набор команд, понятных процессору (архитектура x86, ARM, RISC-V).
- **Примеры:**
  - MOV EAX, 5 (запись числа 5 в регистр),
  - ADD R1, R2 (сложение регистров).
- **Как работает:**
- ISA определяет:
  - Регистры процессора,
  - Формат инструкций,
  - Режимы адресации.
- **Ключевая идея:**
- **Абстракция микроархитектуры** — разные процессоры (например, Intel Core и AMD Ryzen) могут реализовывать одну ISA (x86-64), но по-разному (конвейеры, кэши).
- **Связь с нижними уровнями:**
- ISA — «contract» между программным и аппаратным обеспечением. Производители процессоров (Intel/AMD) гарантируют, что их CPU будут выполнять инструкции ISA, но скрывают детали реализации (уровень 1).
- **Факт:** Документация Intel и AMD (например, Intel SDM ) описывает именно этот уровень. Это основа для разработки компиляторов и ОС.

# Уровень 1: Машинно-зависимое исполнение команд

- **Что это:** Микроархитектура — как именно процессор выполняет инструкции ISA.
- **Примеры:**
  - Конвейеризация (Pentium имел 5-ступенчатый конвейер),
  - Кэш-память L1/L2/L3,
  - Предсказание ветвлений.
- **Как работает:**
  - Процессор разбивает сложные инструкции на микрооперации ( $\mu\text{ops}$ ), оптимизирует порядок выполнения (out-of-order execution), управляет кэшами.
- **Ключевая идея:**
- **Абстракция аппаратной реализации** — программист не знает, как CPU обрабатывает ADD, но ожидает, что результат будет корректен и быстр.
- **Связь с нижними уровнями:**
  - Микроархитектура реализуется на уровне аппаратного обеспечения (уровень 0), но скрыта от программиста.
- **Пример различий:**
  - Intel Core i7 (микроархитектура Skylake) и AMD Ryzen (Zen) реализуют одну ISA (x86-64), но их микроархитектуры принципиально разные.
  - Это объясняет, почему один и тот же код может работать быстрее на CPU одной компании.

# Уровень 0: Аппаратное обеспечение

- **Что это:** Физические компоненты — транзисторы, шины, кристаллы процессора.
- **Примеры:**
  - Логические вентили (AND, OR, NOT),
  - Шины данных и адреса,
  - SRAM-ячейки в кэше.
- **Как работает:**
  - Аппарат преобразует электрические сигналы в вычисления:
    - Транзисторы формируют логические схемы,
    - Тактовая частота синхронизирует операции.
- **Ключевая идея:**
- **Абстракция физических законов** — инженеры проектируют схемы, учитывая законы физики (например, задержки сигналов), но скрывают это от уровня 1.
- **Связь с верхними уровнями:**
  - Аппаратное обеспечение выполняет микрооперации (уровень 1), но не «знает» о существовании программ на Python (уровень 5).
- **Интересно:** Современные процессоры содержат миллиарды транзисторов (например, Apple M2 — 20 млрд), но программисту не нужно об этом знать.

# Важно!

- **Уровень 1 и 0 — не одно и то же:**
  - **Уровень 1** — микроархитектура (логика работы CPU),
  - **Уровень 0** — физические компоненты.
- **Instruction Set Architecture (ISA)**
  - **ISA стандартизирована** (например, x86-64), **но её реализация (микроархитектура) уникальна** для Intel и AMD.
  - Разные процессоры (например, Intel Core и AMD Ryzen) могут реализовывать одну ISA (x86-64), но по-разному (конвейеры, кэши).
- **Приложения не взаимодействуют напрямую с аппаратной частью:**
  - **Современные операционные системы блокируют прямой доступ к оборудованию ради безопасности** (исключение: драйверы в режиме ядра).
  - Приложение → API ОС → Драйвер диска → ISA → Микроархитектура → Аппаратура.
- **Иерархия уровней — это ключ к пониманию компьютеров.** Он показывает, как сложная система строится из простых абстракций, где каждый слой скрывает детали предыдущего. Именно поэтому программист может писать веб-приложения на JavaScript, не зная, как транзисторы складывают числа, а инженер Intel проектирует кэш-память, не думая о том, как работает React.

# Как данные «спускаются» через уровни: **пример**

- **Задача: Вычислить  $2 + 3$  в Python.**
- **1) Уровень 5: Языки высокого уровня**
  - python
  - `result = 2 + 3`
- **2) Уровень 4: Прикладные программы**
  - Интерпретатор Python (приложение) обрабатывает выражение.
- **3) Уровень 3: Операционная система**
  - ОС выделяет память для переменной `result`.
- **4) Уровень 2: Система команд (ISA — Instruction Set Architecture)**
- Компилятор/интерпретатор генерирует машинный код:
  - assembler:
  - `MOV EAX, 2`
  - `ADD EAX, 3`
- **5) Уровень 1: Машинно-зависимое исполнение команд**
  - Процессор разбивает `ADD` на микрооперации, использует АЛУ.
- **6) Уровень 0: Аппаратное обеспечение**
  - Транзисторы в АЛУ выполняют сложение через логические схемы.

# Почему эта модель важна?

- **Упрощение проектирования:**

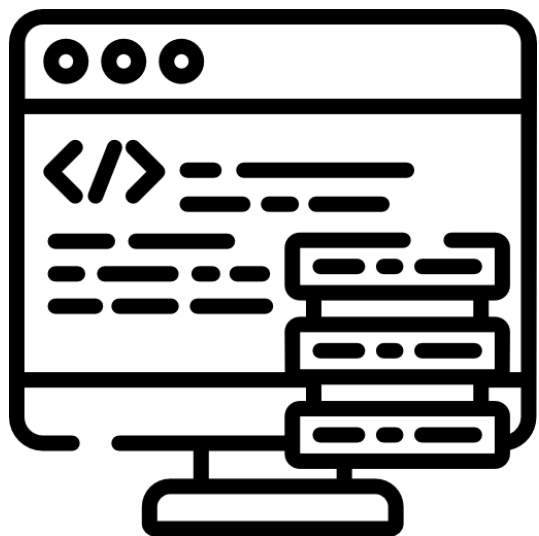
- Каждый уровень можно разрабатывать независимо (например, создавать новые языки без замены процессора).

- **Совместимость:**

- ISA (уровень 2) позволяет запускать одни и те же программы на разных CPU (Intel vs. AMD).

- **Оптимизация:**

- Понимание уровней помогает находить узкие места (например, медленный диск → оптимизация файловой системы, уровень 3).



Пример работы  
простейшей  
программы

# Пример

- Для понимания взаимодействия аппаратного и программного уровней компьютеров, рассмотрим простой пример работы простейшей программы написанной на языке C в операционной системе Linux.

Программа hello на C

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("hello, world\n");
6     return 0;
7 }
```

# Программа hello world на C

Программа hello на C

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("hello, world\n");
6     return 0;
7 }
```

Представление текстового файла hello.c в кодах ASCII

|     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| #   | i   | n   | c   | l   | u   | d   | e   | SP  | <   | s   | t   | d   | i   | o   | .   |
| 35  | 105 | 110 | 99  | 108 | 117 | 100 | 101 | 32  | 60  | 115 | 116 | 100 | 105 | 111 | 46  |
| h   | >   | \n  | \n  | i   | n   | t   | SP  | m   | a   | i   | n   | (   | )   | \n  | {   |
| 104 | 62  | 10  | 10  | 105 | 110 | 116 | 32  | 109 | 97  | 105 | 110 | 40  | 41  | 10  | 123 |
| \n  | SP  | SP  | SP  | SP  | p   | r   | i   | n   | t   | f   | (   | "   | h   | e   | l   |
| 10  | 32  | 32  | 32  | 32  | 112 | 114 | 105 | 110 | 116 | 102 | 40  | 34  | 104 | 101 | 108 |
| l   | o   | ,   | SP  | w   | o   | r   | l   | d   | \   | n   | "   | )   | ;   | \n  | SP  |
| 108 | 111 | 44  | 32  | 119 | 111 | 114 | 108 | 100 | 92  | 110 | 34  | 41  | 59  | 10  | 32  |
| SP  | SP  | SP  | r   | e   | t   | u   | r   | n   | SP  | 0   | ;   | \n  | }   | \n  |     |
| 32  | 32  | 32  | 114 | 101 | 116 | 117 | 114 | 110 | 32  | 48  | 59  | 10  | 125 | 10  |     |

**Программа hello** **начинает свой жизненный цикл как исходная программа** (или файл с исходным кодом), которую программист создает с помощью текстового редактора и сохраняет в файле с именем hello.c.

**Исходный код программы представляет собой последовательность битов**, каждый из которых принимает значение 0 или 1, организованных в 8-битные блоки, получившие название байты. Каждый байт в исходном коде представляет некоторый символ.

Большинство компьютерных систем представляют текстовые символы в стандарте ASCII (American Standard Code for Information Interchange – американский стандартный код обмена информацией), согласно которому каждый символ представляется уникальным однобайтным целым числом 1.

Например, в листинге - программа hello.c показана в кодах ASCII.

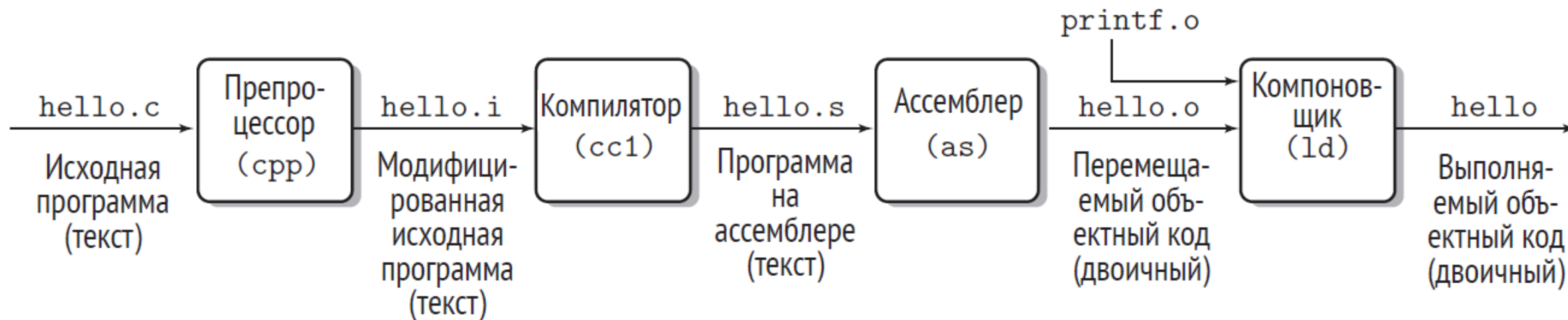
В настоящее время большинство текстовых файлов уже хранится в кодировке Unicode (UTF-8, UTF-16, и т.д.)

# Информация – это биты + контекст

- Программа `hello` хранится в файле как последовательность байтов.
- Каждый байт принимает целочисленное значение, соответствующее некоторому символу.
- Например, первый байт имеет значение 35, которому соответствует символ «#». Второй байт имеет значение 105, которому соответствует символ «i», и т. д. Обратите внимание, что текстовая строка заканчивается невидимым символом перевода строки «\n», который представлен целым значением 10. Такие файлы, как `hello.c`, содержащие исключительно символы, называются текстовыми файлами, а все другие – двоичными файлами.
- Представление файла `hello.c` иллюстрирует одну из фундаментальных идей. **Вся информация в системе, включая файлы на дисках, программы и данные пользователей, хранящиеся в памяти, а также данные, передаваемые по сети, представляется в виде битовых блоков.**
- Единственное, что отличает разные виды данных друг от друга, – контекст, в котором мы их рассматриваем. Например, в разных контекстах одна и та же последовательность байтов может представлять целое число, число с плавающей точкой, строку символов или машинную инструкцию.
- Как программисты мы должны понимать машинное представление чисел, поскольку они не тождественны целым или вещественным числам. Они суть конечные приближения, которые могут вести себя непредсказуемым образом.

# Программа hello

- **Программа hello начинает свою жизнь как программа на языке высокого уровня**, поскольку в этой форме она может быть прочитана и понята человеком.
- Однако, чтобы запустить программу `hello.c` в системе, операторы на языке C должны быть преобразованы другими программами в некоторую последовательность инструкций на машинном языке низкого уровня.
- Эти инструкции затем упаковываются в выполняемую объектную программу и сохраняются в двоичном файле на диске.
- Объектные программы также называются выполняемыми объектными файлами.
- Далее компилятор читает исходный файл `hello.c` и транслирует его в выполняемый объектный файл `hello.o`.
- Трансляция выполняется в четыре этапа, как показано на рисунке.
- **Совокупность программ, выполняющих эти четыре этапа (препроцессор, компилятор, ассемблер и компоновщик), называется системой компиляции.**



Система компиляции

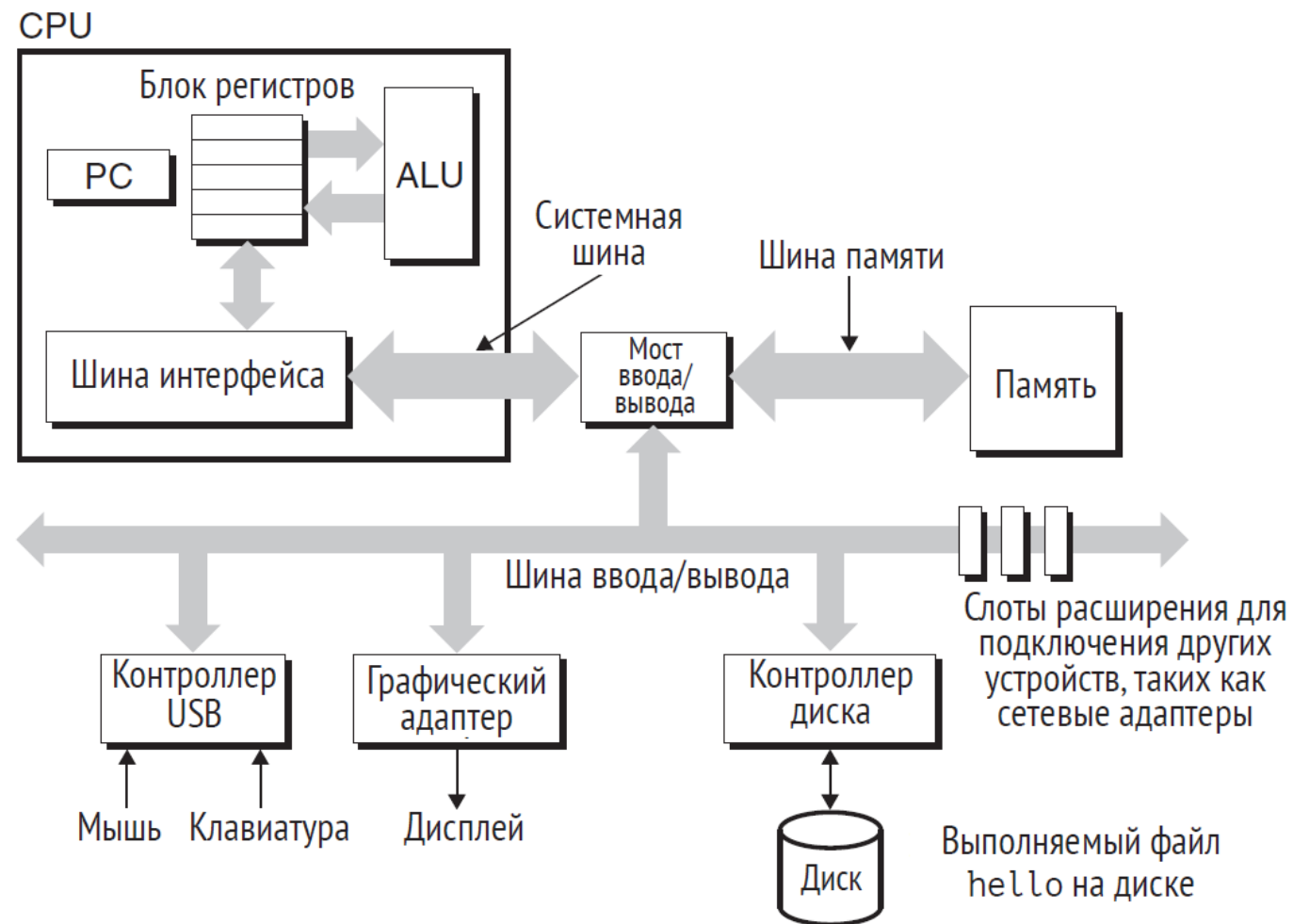
# Система компиляции

- **Совокупность программ, выполняющих эти четыре этапа (препроцессор, компилятор, ассемблер и компоновщик), называется системой компиляции.**
- **Этап препроцессора** (или этап предварительной обработки). Препроцессор (cpp) изменяет исходную программу в соответствии с директивами, которые начинаются с символа «#». Например, директива `#include <stdio.h>` в строке 1 программы `hello.c` заставляет препроцессор прочитать содержимое системного заголовочного файла `stdio.h` и вставить его непосредственно в текст программы. В результате получается другая программа на языке C, обычно с расширением `.i`.
- **Этап компиляции.** Компилятор (cc) транслирует текстовый файл `hello.i` в текстовый файл `hello.s`, который содержит программу на языке ассемблера. Эта программа включает следующее определение функции `main`:

```
1 main:
2     subq $8, %rsp
3     movl $.LC0, %edi
4     call puts
5     movl $0, %eax
6     addq $8, %rsp
7     ret
```
- Каждый оператор в строках 2–7 этого определения точно описывает одну из инструкций низкоуровневого машинного языка в текстовой форме. Польза языка ассемблера прежде всего в том, что он представляет общий выходной язык для компиляторов разных языков высокого уровня. Например, компиляторы языка C и языка Fortran генерируют выходные файлы на одном и том же языке ассемблера.
- **Этап ассемблирования.** Ассемблер (as) транслирует файл `hello.s` в машинные инструкции, упаковывает их в форму, известную как перемещаемый объектный код, и запоминает результат в объектном файле `hello.o`. Файл `hello.o` – это двоичный файл, содержащий 17 байт, которые кодируют машинные инструкции, составляющие функцию `main`. Если открыть `hello.o` в текстовом редакторе, то вы увидите совершенно непонятную абракадабру.
- **Этап компоновки.** Обратите внимание, что наша программа `hello` вызывает функцию `printf` из стандартной библиотеки C, которая поставляется в комплекте с любым компилятором языка C. Функция `printf` находится в отдельном предварительно скомпилированном объектном файле с именем `printf.o`, который тем или иным способом должен быть объединен с нашей программой `hello.o`. Это объединение осуществляет компоновщик (ld). В результате получается выполняемый объектный файл (или просто выполняемый файл), готовый к загрузке и выполнению системой.

# Аппаратная организация системы

- Чтобы понять, что происходит с программой hello во время выполнения, мы должны иметь представление о том, как устроена аппаратная часть типичной вычислительной системы, блок-схема которой показана на рисунке.
- Эта конкретная блок-схема отражает устройство современных систем Intel, однако примерно такое же имеют все вычислительные системы.



Аппаратная организация типичной вычислительной системы.  
CPU: центральное процессорное устройство или просто процессор,  
ALU: арифметико-логическое устройство, PC (program counter): счетчик команд,  
USB (Universal Serial Bus): универсальная последовательная шина

# Аппаратная организация системы

- **Шины.** Вычислительную систему пронизывает совокупность электрических проводников, так называемых шин, по которым байты информации циркулируют между компонентами системы. Обычно шины конструируются таким образом, чтобы по ним можно было передавать байты порциями фиксированного размера, которые называют словами. Число байтов в слове (размер слова) является одним из фундаментальных параметров, которые изменяются от системы к системе. В большинстве современных систем используются слова с размером 4 байта (32 бита) или 8 байт (64 бита).
- **Устройства ввода/вывода** – это средства связи с внешним миром. В нашем примере системы имеется четыре устройства ввода/вывода: клавиатура и мышь для ввода данных пользователем, устройство для отображения данных пользователю и дисковое устройство (или просто диск) для долгосрочного хранения данных и программ. В начальный момент выполняемый файл хранится на диске.
- **Каждое устройство ввода/вывода подключено к шине ввода/вывода посредством контроллера или адаптера. Различие между ними заключается в их конструктивных особенностях.**
- **Контроллеры** – это платы с наборами микросхем, установленные в самом устройстве или на главной печатной плате (ее еще часто называют материнской платой).
- **Адаптер** – это плата, которая подключается через контактное гнездо на материнской плате. Независимо от конструкции таких устройств, их назначение заключается в том, чтобы передавать информацию между шиной и устройством ввода/вывода в обоих направлениях. В главе 6 более подробно описана работа таких устройств ввода/вывода, как диск.

# Аппаратная организация системы

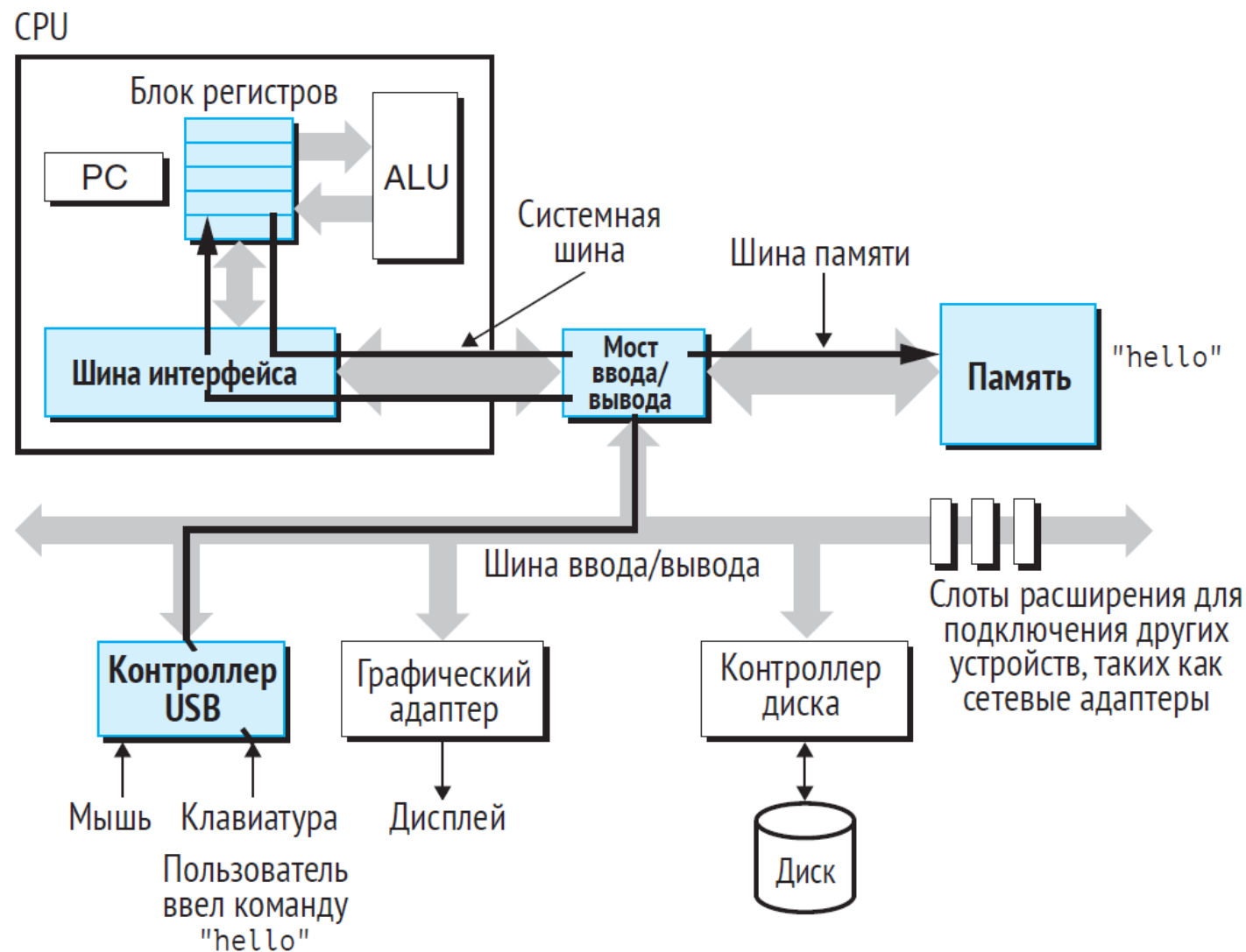
- **Основная память** – временное хранилище, в котором находятся сама программа и данные, которыми она манипулирует во время выполнения. Физически основная память состоит из совокупности микросхем динамической памяти с произвольным доступом (Dynamic Random Access Memory, DRAM). Логически основная память организована в виде линейного массива байтов, каждый из которых имеет свой уникальный адрес (индекс элемента массива); отсчет адресов начинается с нуля. Машинные инструкции, составляющие программу, могут состоять из разного числа байтов. Размер элементов данных, соответствующих переменным в программах на C, зависит от их типов. Например, на машине x86\_64, работающей под Linux, тип данных short требует двух байт, типы int и float – четырех байт, а типы long и double – восьми байт.
- **Центральный процессор** (ЦП; Central Processing Unit, CPU), или просто процессор, – это механизм, который интерпретирует (или выполняет) инструкции, хранящиеся в основной памяти. Его ядро составляет устройство памяти с емкостью в одно слово (или регистр) – счетчик команд (Program Counter, PC). В любой конкретный момент времени он хранит адрес некоторой машинной инструкции в основной памяти.
- С момента включения системы и до ее выключения процессор снова и снова выполняет инструкцию, на которую указывает счетчик команд, и затем обновляет значение счетчика, выполняя переход к следующей инструкции. Кажется, что процессор работает в соответствии с очень простой моделью выполнения инструкций, определяемой его архитектурным набором команд. Согласно этой модели инструкции выполняются в строгой последовательности, а выполнение одной инструкции происходит в несколько этапов. Сначала процессор читает инструкцию из памяти по адресу в счетчике команд (PC), интерпретирует биты инструкции, выполняет простые операции, определяемые инструкцией, а затем обновляет значение счетчика, записывая в него адрес следующей инструкции, которая может размещаться за текущей или где-то в другом месте в памяти.

# Аппаратная организация системы

- Существует всего несколько таких простых операций, и все они связаны с обслуживанием основной памяти, блока регистров и арифметико-логического устройства (Arithmetic/Logic Unit, ALU).
- **Блок регистров** – небольшое запоминающее устройство из совокупности регистров, каждый из которых имеет свое уникальное имя и может хранить одно слово. Устройство ALU вычисляет новые значения данных и адресов.
- Назовем лишь несколько примеров простых операций, которые процессор может выполнить по требованию той или иной инструкции:
  - **загрузка:** копирует байт или слово из основной памяти в регистр, затирая при этом предыдущее содержимое этого регистра;
  - **сохранение:** копирует байт или слово из регистра в некоторую ячейку основной памяти, затирая при этом предыдущее содержимое этой ячейки;
  - **выполнение** арифметико-логической операции: копирует содержимое двух регистров в ALU, выполняет соответствующую операцию с этими словами и запоминает результат в одном из регистров, затирая при этом предыдущее содержимое этого регистра;
  - **переход:** извлекает слово из самой инструкции и копирует его в счетчик команд (PC), затирая предыдущее его содержимое.
- Мы говорим «кажется, что процессор работает в соответствии с очень простой моделью, определяемой его архитектурным набором», но на самом деле механика работы современных процессоров намного сложнее, чем было сказано ранее.
- Поэтому мы будем различать архитектурный набор команд процессора, определяющий эффект каждой машинной инструкции, и его микроархитектуру, определяющую фактическую реализацию процессора.

# Выполнение программы hello

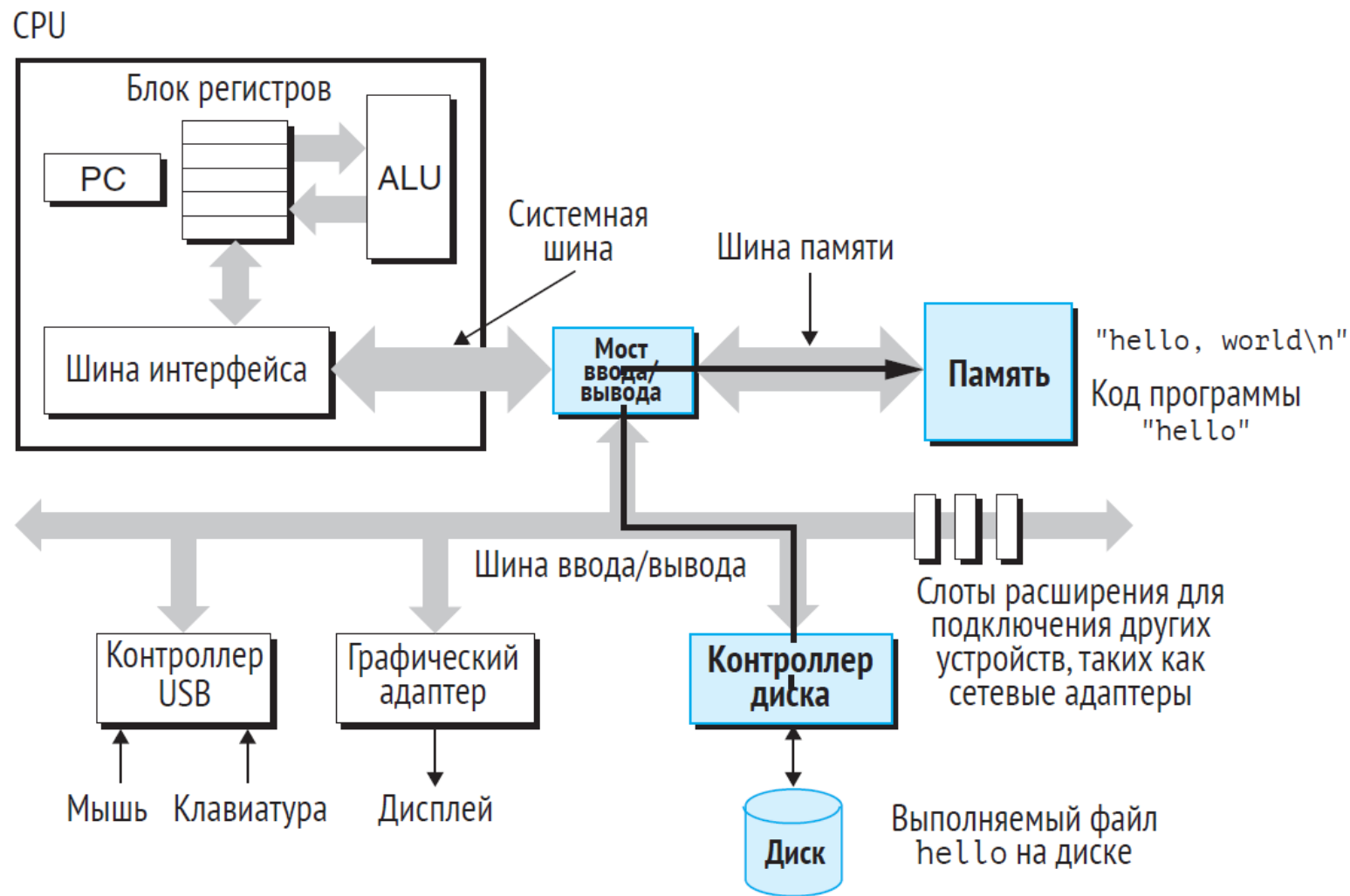
- Ознакомившись с простым описанием аппаратной организации и операций, мы начинаем понимать, что происходит, когда мы запускаем наш пример программы.
- Здесь мы должны опустить множество деталей, которые учтем позже, а пока нас вполне удовлетворит общая картина.
- Сначала свои инструкции выполнит командная оболочка, ожидающая, когда мы введем команду с клавиатуры.
- Как только мы введем символы ./hello, командная оболочка прочитает каждый из них в регистр и сохранит в основной памяти, как показано на рисунке.



Чтение команды hello с клавиатуры

# Выполнение программы hello

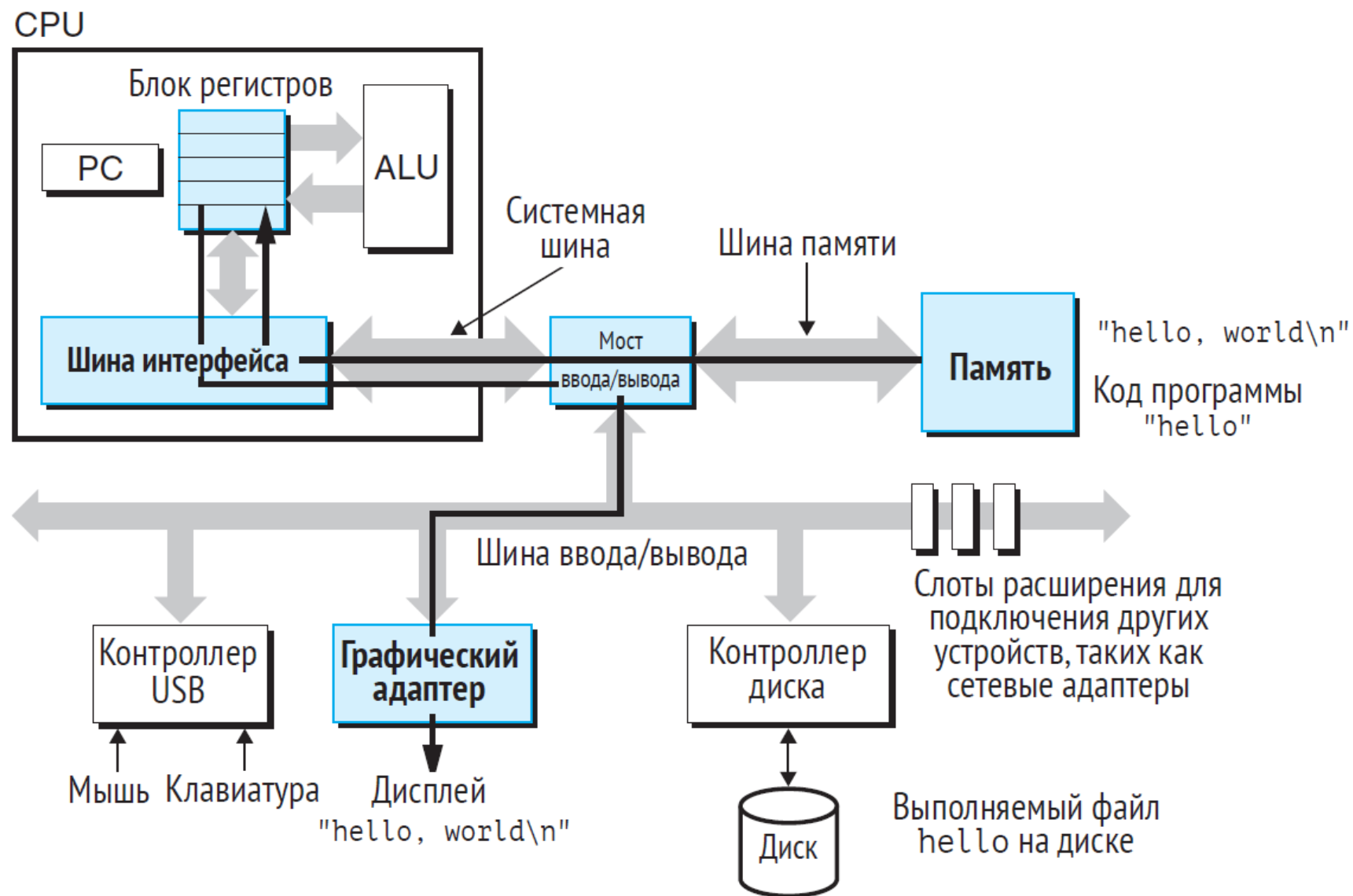
- Когда мы нажмем клавишу Enter, оболочка воспримет это как сигнал окончания ввода команды.
- После этого она загрузит выполняемый файл hello, осуществив последовательность инструкций, которая скопирует в основную память программные коды и данные, содержащиеся в объектном файле hello на диске.
- Данные включают строку символов "hello, world\n", которая в конечном итоге будет выведена на экран.
- Используя метод, известный как прямой доступ к памяти (Direct Memory Access, DMA), данные перемещаются с диска в оперативную память, минуя процессор.
- Этот шаг показан на рисунке.



Загрузка выполняемого файла в основную память

# Выполнение программы hello

- Как только код и данные из объектного файла hello будут загружены в память, процессор начинает выполнять машинные инструкции подпрограммы main в программе hello.
- Эти инструкции копируют байты строки "hello, world\n" из основной памяти в регистры, а оттуда – на дисплей, на экране которого они затем отображаются.
- Эти этапы показаны на рисунке



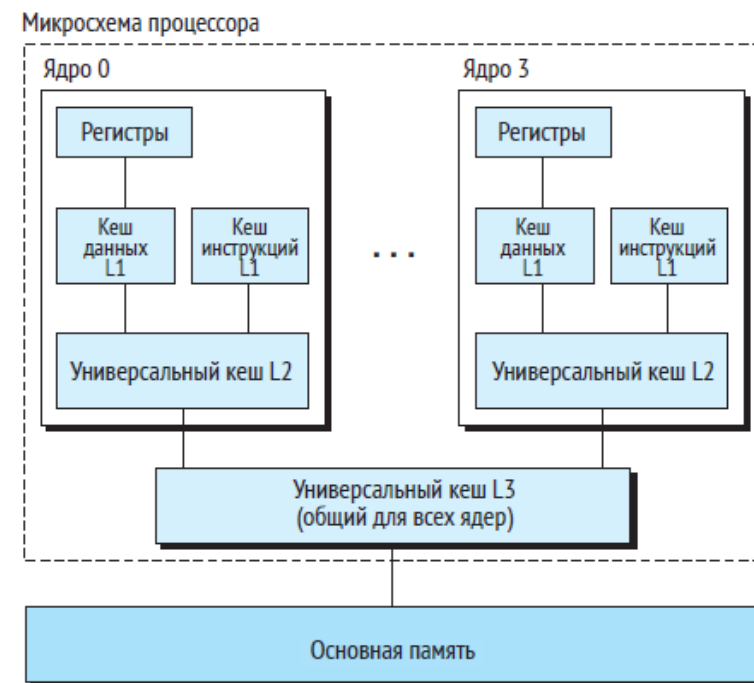
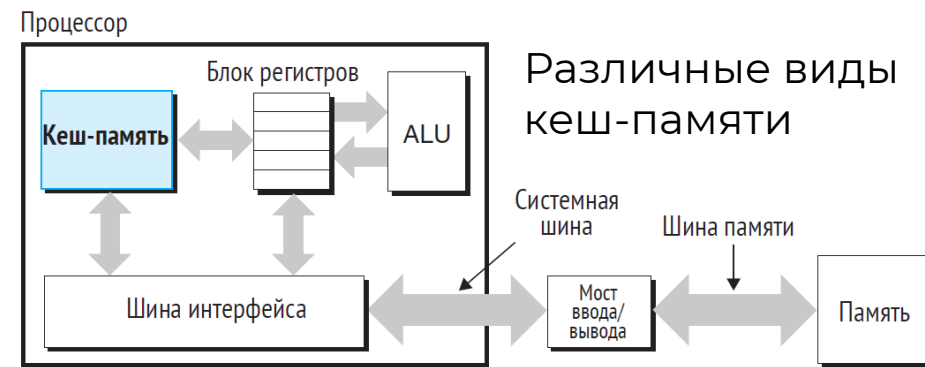
Вывод строки из основной памяти на экран дисплея

# Различные виды кеш-памяти

- Важный урок из этого простого примера заключается в том, что система затрачивает уйму времени на перемещение информации из одного места в другое. Машинные инструкции в программе `hello` в начальный момент хранятся на диске. Когда производится загрузка программы, они копируются в основную память.
- По мере выполнения программы ее инструкции копируются из основной памяти в процессор. Аналогично строка данных `"hello, world\n"`, которая первоначально хранилась на диске, копируется в основную память, а затем из основной памяти на устройство отображения.
- С точки зрения программиста такой большой объем копирования ложится тяжким бременем на систему и существенно снижает «истинную производительность» программы. Следовательно, главная цель системных проектировщиков заключается в том, чтобы максимально ускорить операции копирования данных.
- В силу физических законов чем больше запоминающее устройство, тем медленнее оно работает. И в то же время создание быстродействующих запоминающих устройств обходится дороже, чем более медленных.
- Например, емкость диска может оказаться в 1000 раз больше объема оперативной памяти, но, чтобы прочесть слово с диска, требуется в 10 млн раз больше времени, чем из основной памяти.
- Аналогично типичный блок регистров может хранить лишь несколько сотен байтов информации, в то время как основная память – миллиарды. Однако процессор способен читать данные из регистров примерно в 100 раз быстрее, чем из основной памяти. Более того, по мере развития полупроводниковых технологий расхождения в скорости доступа к регистрам и к основной памяти продолжают углубляться. Увеличить быстродействие процессора намного проще, чем заставить основную память работать быстрее.

# Различные виды кеш-памяти

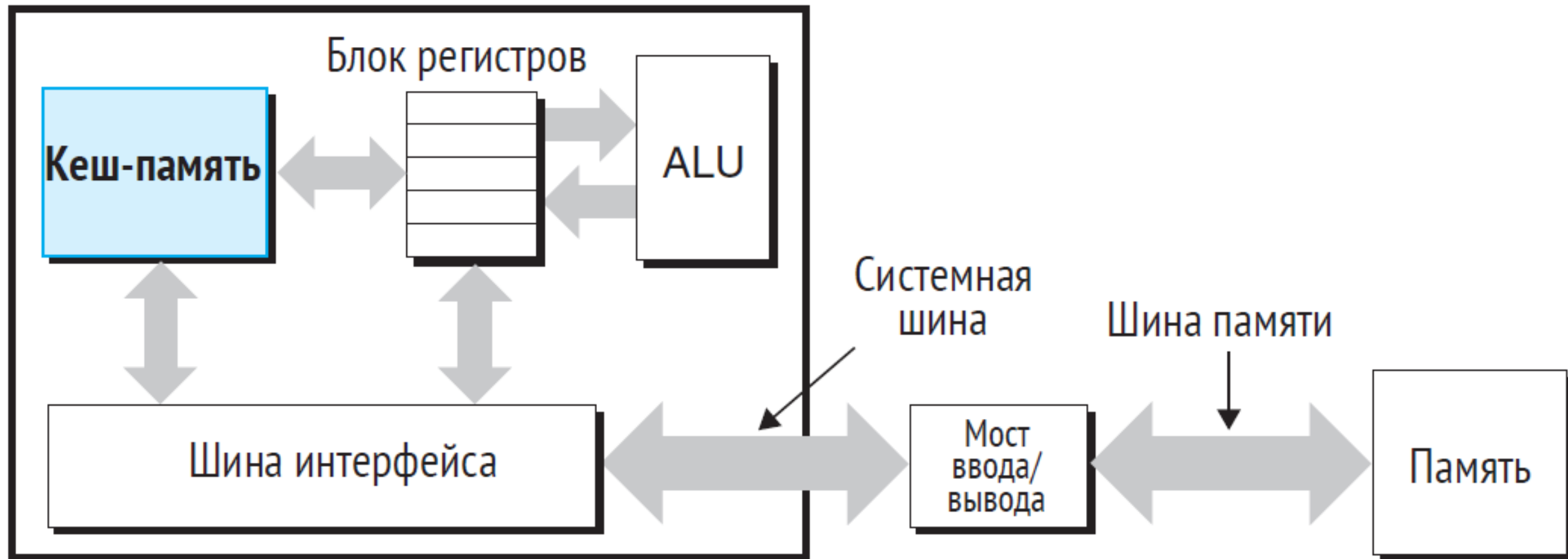
- Чтобы уменьшить разрыв между процессором и основной памятью, создатели процессоров включили в них небольшие быстродействующие устройства хранения, получившие название **кеш-память** (или просто кеш) и служащие для временного хранения информации, которая, возможно, потребуется процессору в ближайшем будущем.
- На рисунке **показана типичная система с кеш-памятью**.
- **Кеш L1** (его еще называют кешем первого уровня) внутри процессора может хранить десятки тысяч байт, и доступ к ним осуществляется почти так же быстро, как к регистрам. Еще больший объем имеет **кеш L2** (кеш второго уровня) – он может вместить от сотен тысяч до миллионов байт. Этот кеш связан с процессором специальной шиной.
- **Скорость доступа к кешу L2 в 5 раз ниже скорости доступа к кешу L1**, и все равно она в 5–10 раз выше скорости доступа к основной памяти. Кешы L1 и L2 построены по технологии, известной как статическая память с произвольным доступом (Static Random Access Memory, SRAM).
- Более новые и более мощные системы имеют три уровня кешей: L1, L2 и L3.
- Идея кеширования состоит в том, чтобы дать системе возможность получить положительный эффект от наличия большого объема памяти и очень короткого времени доступа за счет улучшения локальности – тенденции программ использовать данные и код, находящиеся в ограниченных областях памяти. Использование кешей для хранения данных, к которым программа, вероятно, будет часто обращаться, позволяет выполнять большинство операций с памятью, используя быстрые кешы.
- **Один из самых важных уроков заключается в том, что прикладные программисты, которые знают о наличии кеш-памяти, могут воспользоваться ею и увеличить производительность своих программ на порядок.**



Организация многоядерного процессора.  
Четыре процессорных ядра размещены на одном кристалле

# Различные виды кеш-памяти

Процессор



# Устройства памяти образуют иерархию

- Идея поместить небольшое, зато более быстрое запоминающее устройство (кеш-память) между процессором и более емким, но с худшим быстродействием запоминающим устройством (основной памятью) оказалась весьма плодотворной.
- Фактически запоминающие устройства в любой вычислительной системе образуют иерархию, подобную изображенной на рисунке.
- По мере движения по этой иерархии сверху вниз устройства становятся все медленнее, объемнее, а стоимость хранения одного байта уменьшается.
- Регистры находятся на вершине иерархии, которая обозначается как уровень 0 (L0).
- Кеш-память занимает уровни с 1 по 3 (L1–L3).
- Основная память находится на уровне 4 (L4) и т. д.
- Основная идея иерархии памяти заключается в том, что память одного уровня служит кешем для следующего нижнего уровня. То есть блок регистров – это кеш для кеш-памяти L1.
- Кеши L1 и L2 служат кешами для кеш-памяти L2 и L3 соответственно.
- Кеш L3 служит кешем для основной памяти, а та, в свою очередь, – кешем для диска.
- В некоторых сетевых системах с распределенными файловыми системами локальный диск служит кешем для данных, хранящихся на дисках других систем.



# Пример иерархии памяти



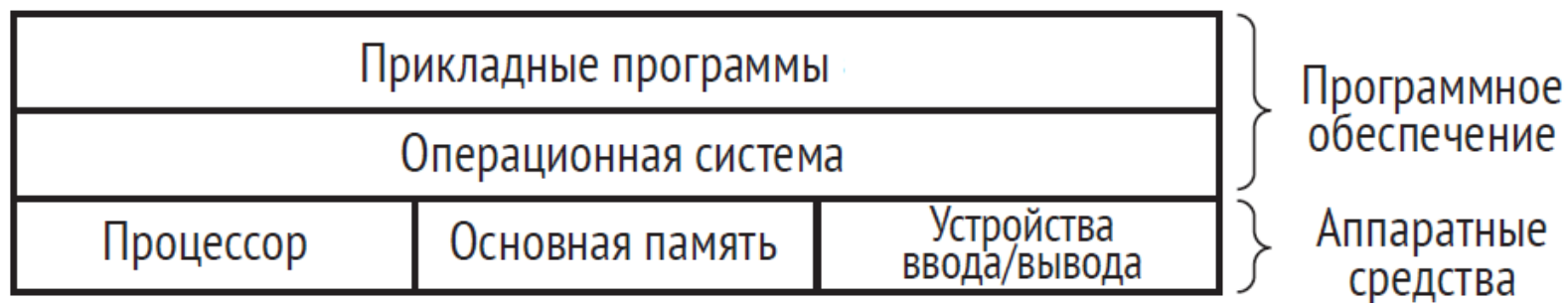
# Повсеместное кеширование в современных компьютерных системах

| Тип                 | Что кеширует           | Где кеширует                      | Задержка<br>(в циклах) | Управляется             |
|---------------------|------------------------|-----------------------------------|------------------------|-------------------------|
| Регистры процессора | 4- или 8-байтные слова | Регистры в процессоре             | 0                      | Компилятором            |
| TLB                 | Таблицы адресов        | В буфере ассоциативной трансляции | 0                      | Аппаратным модулем MMU  |
| Кеш L1              | 64-байтные блоки       | Аппаратный кеш L1                 | 4                      | Аппаратно               |
| Кеш L2              | 64-байтные блоки       | Аппаратный кеш L2                 | 10                     | Аппаратно               |
| Кеш L3              | 64-байтные блоки       | Аппаратный кеш L3                 | 50                     | Аппаратно               |
| Виртуальная память  | Страницы по 4 Кбайт    | Основная память                   | 200                    | Аппаратно + OS          |
| Буферный кеш        | Фрагменты файлов       | Основная память                   | 200                    | OS                      |
| Дисковый кеш        | Секторы диска          | Контроллер диска                  | 100 000                | Аппаратурой контроллера |
| Сетевой кеш         | Фрагменты файлов       | Локальный диск                    | 10 000 000             | Клиент NFS              |
| Кеш браузера        | Веб-страницы           | Локальный диск                    | 10 000 000             | Веб-браузером           |
| Веб-кеш             | Веб-страницы           | Диски на удаленных серверах       | 1 000 000 000          | Прокси-сервером         |

**Аббревиатуры:** **TLB:** Translation Lookaside Buffer (буфер ассоциативной трансляции); **MMU:** Memory Management Unit (блок управления памятью); **OS:** Operating System (операционная система); **NFS:** Network File System (сетевая файловая система)

# ОС управляет работой аппаратных средств

- Вернемся к нашей программе hello. Когда оболочка загружала и запускала программу hello и когда программа hello отображала свое сообщение, ни та ни другая программа не обращалась напрямую к клавиатуре, диску или основной памяти.
- Для этого они пользовались услугами, предоставляемыми операционной системой.
- Операционную систему можно представить как некоторый слой программного обеспечения между прикладной программой и аппаратными средствами, как показано на рисунке.

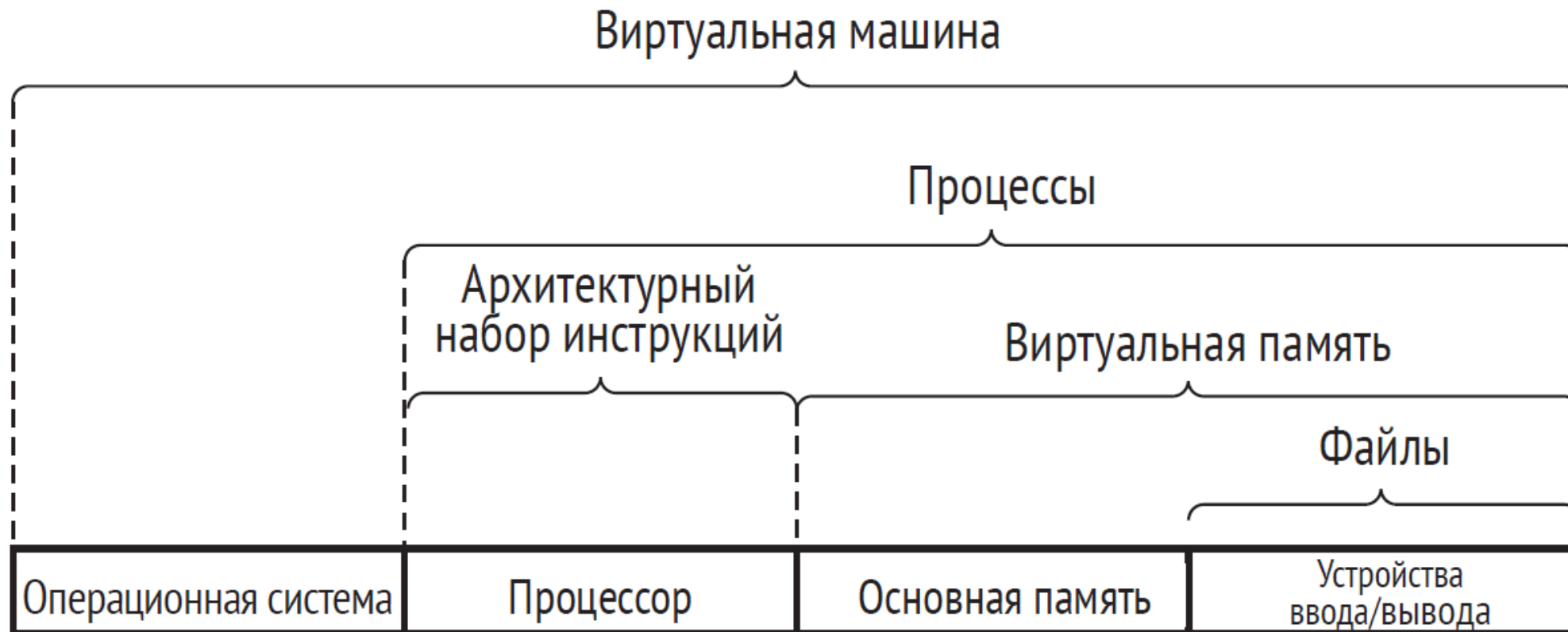


Многослойная организация компьютерной системы

- Любые операции с аппаратными средствами прикладные программы должны выполнять через операционную систему.
- Операционная система прежде всего должна отвечать двум основным требованиям: (1) защищать аппаратные средства от катастрофических действий вышедшей из-под контроля программы и (2) предоставлять приложениям простые и единообразные механизмы манипулирования сложными и часто сильно отличающимися низкоуровневыми аппаратными средствами.
- **Для этого операционная система предлагает набор фундаментальных абстракций: процессы, виртуальную память и файлы.**

# ОС управляет работой аппаратных средств

- Как видно по рисунку, **файлы** являются абстракцией устройств ввода/вывода, **виртуальная память** является абстракцией как основной памяти, так и дисковых устройств ввода/вывода, а **процессы** – абстракцией процессора, основной памяти и устройств ввода/вывода.

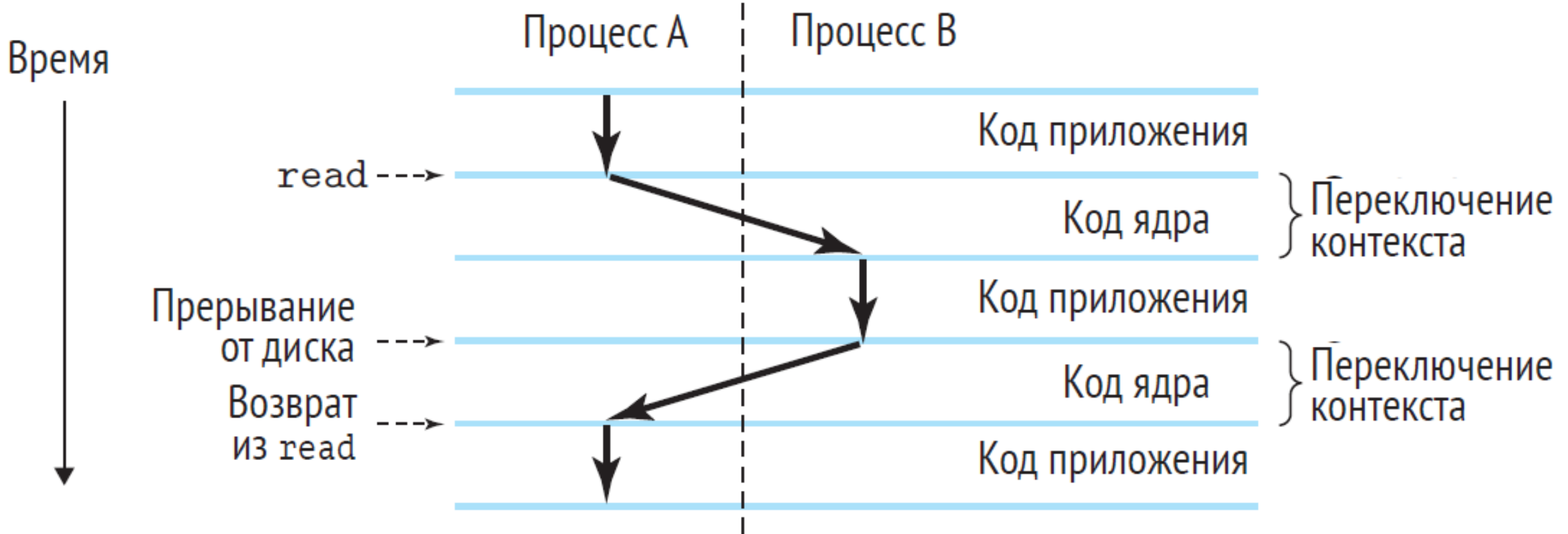


Некоторые абстракции, предоставляемые компьютерной системой.  
Основная задача компьютерных систем – обеспечить абстрактные представления на разных уровнях, чтобы скрыть сложность фактических реализаций

# Процессы

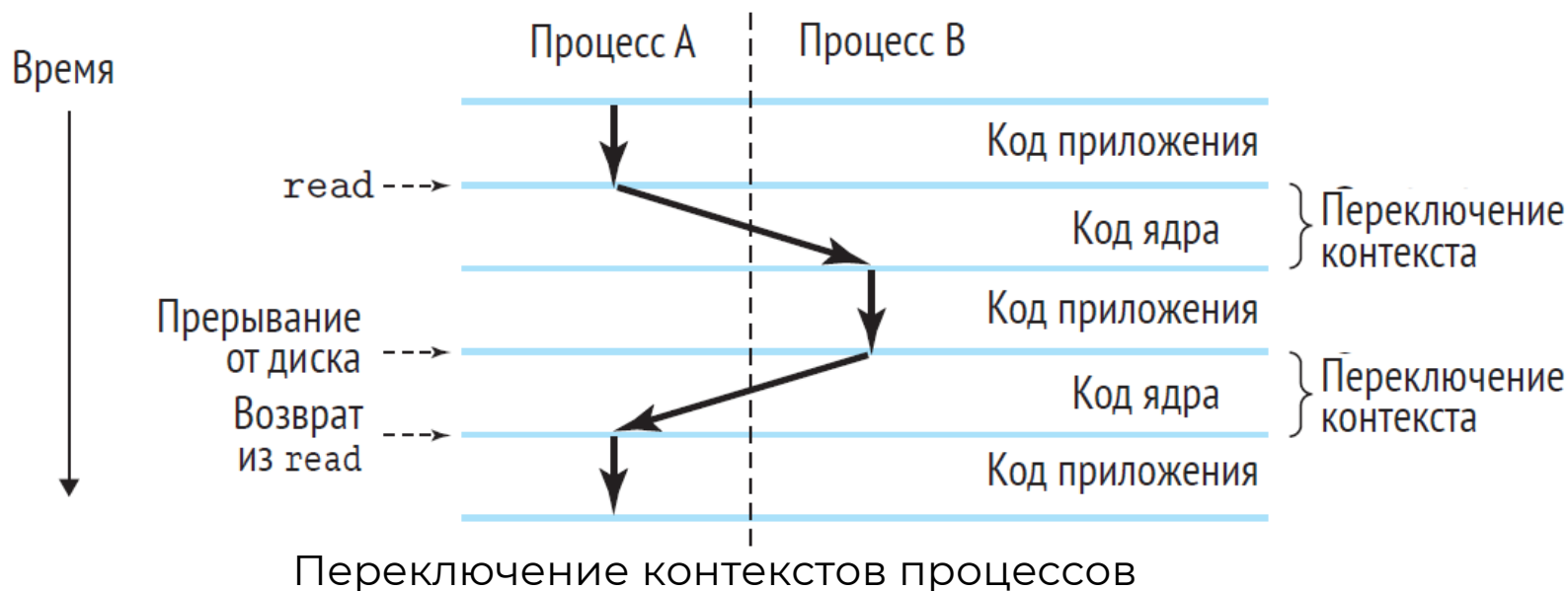
- Когда программа, такая как **hello**, запускается в современной системе, операционная система создает иллюзию, что она – единственная программа, выполняющаяся в системе. С точки зрения программы создается впечатление, что только она распоряжается процессором, основной памятью и устройствами ввода/вывода. **Процессор как бы выполняет инструкции программы подряд, одну за другой, без прерываний, и только код программы и ее данные являются единственными объектами, пребывающими в памяти системы.** Источником таких иллюзий является понятие процесса, одной из наиболее важных и успешных идей в теории вычислительных машин и систем.
- **Процесс – это абстракция операционной системы, представляющая выполняемую программу.**
- В одной и той же системе одновременно может выполняться множество процессов, и в то же время каждому процессу кажется, что только он пользуется аппаратными средствами. Под одновременным (или параллельным) выполнением мы понимаем поочередное выполнение инструкций то одного, то другого процесса. В большинстве систем существует больше процессов, выполняющихся одновременно, чем процессоров, на которых они выполняются.
- Традиционные системы могут выполнять только одну программу в каждый конкретный момент времени, тогда как новые многоядерные процессоры способны одновременно выполнять программный код нескольких программ. В любом случае может показаться, что один процессор реализует несколько процессов одновременно, если будет переключаться между ними очень быстро.
- Операционная система проделывает такое чередование с помощью механизма переключения контекста. Чтобы упростить остальную часть этого обсуждения, мы будем рассматривать только однопроцессорную систему с единственным процессором.

# Переключение контекстов процессов



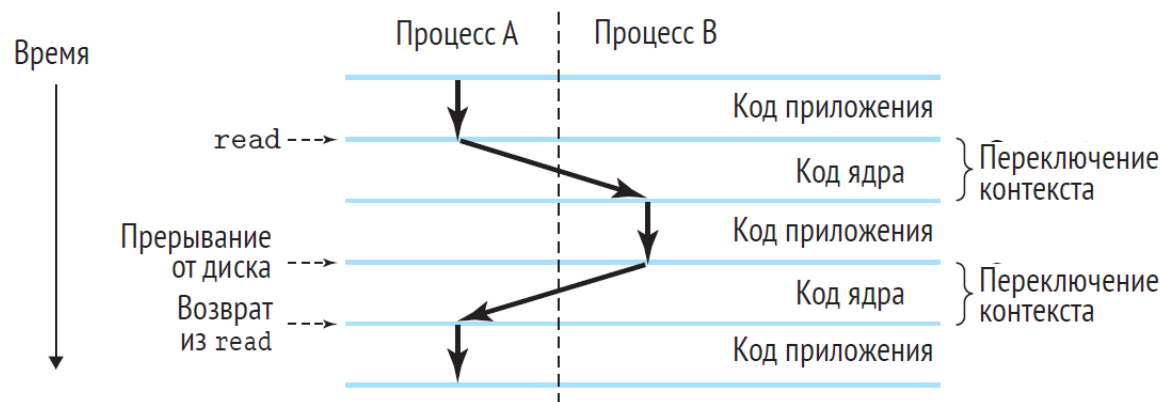
# Процессы

- Операционная система хранит всю информацию о состоянии, необходимую для правильного выполнения процесса.
- Состояние, известное как **контекст**, содержит такие сведения, как текущее значение счетчика команд РС, состояние блока регистров и содержимое основной памяти.
- В любой конкретный момент времени в системе выполняется только один процесс.
- Когда операционная система принимает решение передать управление некоторому другому процессу, она производит переключение контекста, запоминая контекст текущего процесса и восстанавливая контекст нового, после чего передает управление новому процессу.
- Новый процесс возобновляет выполнение точно с того места, в котором он был прерван.
- Эту идею иллюстрирует рисунок на примере нашей программы hello.



# Процессы

- В рассматриваемом нами примере существует два процесса: процесс командной оболочки и процесс hello.
- Первоначально система выполняет только один процесс, а именно процесс командной оболочки, которая ожидает ввода команды. Когда мы обращаемся к нему с требованием запустить программу hello, оболочка выполняет наше требование, вызывая специальную функцию, так называемый системный вызов, который передает управление операционной системе.
- Операционная система сохраняет контекст процесса оболочки, создает новый процесс hello с его контекстом, а затем передает управление новому процессу hello.
- После завершения hello операционная система восстанавливает контекст процесса оболочки и возвращает ему управление, а он затем ждет ввода следующей команды.



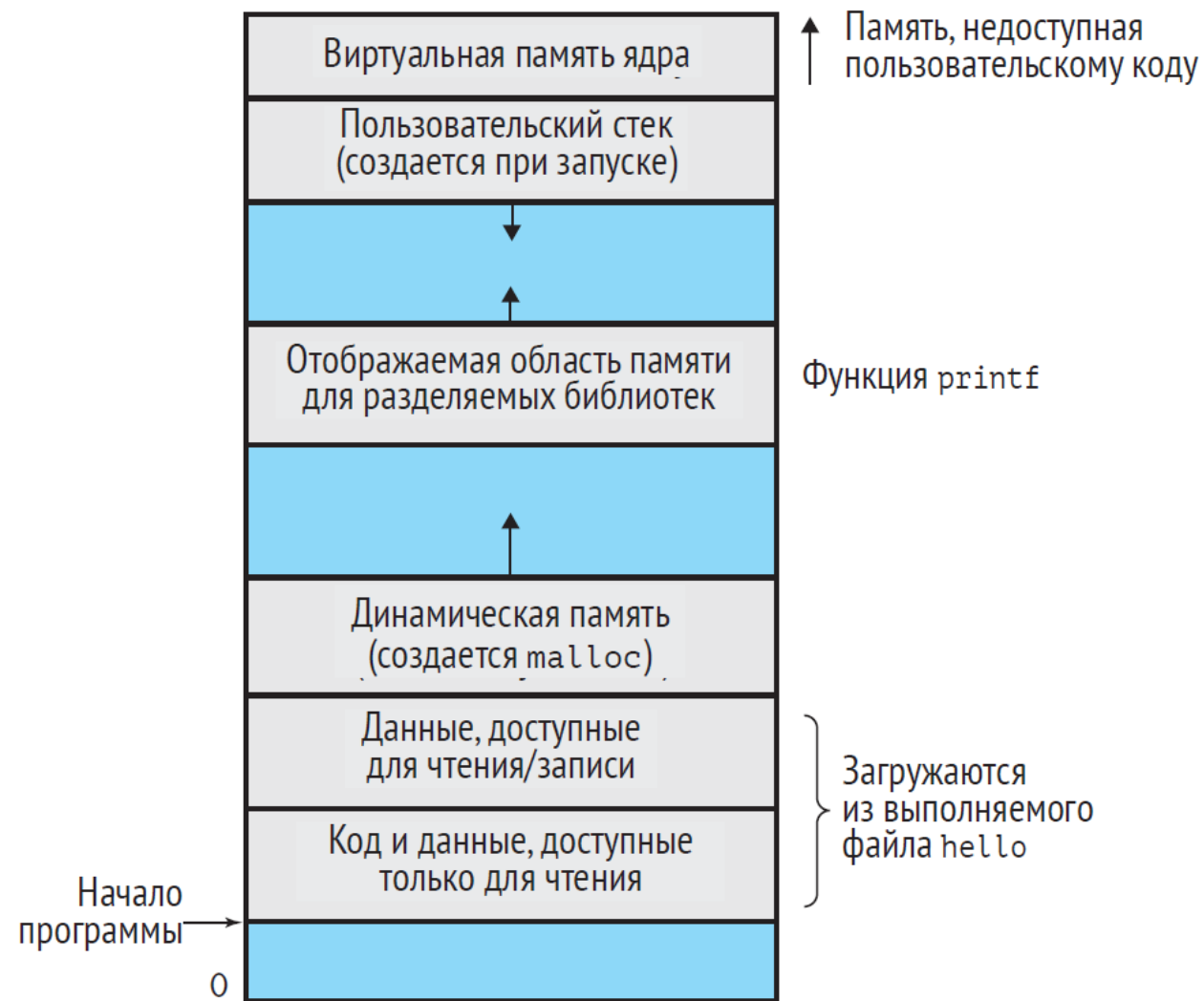
- Как показано на рисунке, переключение с одного процесса на другой производится ядром операционной системы.
- Ядро – это часть кода операционной системы, которая всегда находится в памяти.**
- Когда прикладная программа требует от операционной системы какого-либо действия, например чтения из файла или записи в файл, она выполняет специальную инструкцию системного вызова, передавая управление ядру.
- Затем ядро выполняет запрошенную операцию и возвращает управление прикладной программе. Обратите внимание, что **ядро – это не отдельный процесс, а блок кода и структур данных, которые система использует для управления всеми процессами.**
- Реализация абстракции процесса требует тесного взаимодействия аппаратных и программных средств операционной системы.**

# Потоки

- Обычно мы представляем себе процесс как единственный поток управления, однако в современных системах процесс может состоять из множества единиц выполнения, которые называют **потоками** выполнения (или нитями), каждый поток выполняется в контексте процесса, использует тот же программный код и глобальные данные, что и сам процесс.
- Роль потоков как программных моделей непрерывно возрастает в связи с требованиями к поддержке параллельных вычислений, предъявляемыми сетевыми серверами, поскольку организовать совместное использование данных несколькими потоками намного проще, чем несколькими процессами, а также в силу того, что потоки обычно значительно эффективнее процессов.
- Кроме того, многопоточность является одним из способов ускорить работу программ, когда в системе имеется несколько процессоров,

# Виртуальная память

- **Виртуальная память** – это абстракция, которая порождает в каждом процессе иллюзию, что лишь он один использует основную память. Каждый процесс имеет одно и то же представление о памяти, которое известно как его виртуальное адресное пространство.
- Виртуальное адресное пространство для процессов операционной системы Linux показано на рисунке. (Другие Unix-системы используют ту же топологию.)
- В системе Linux верхняя часть адресного пространства резервируется для программного кода и данных операционной системы, которые являются общими для всех процессов.
- В нижней части адресного пространства находятся программный код и данные, принадлежащие процессам пользователей.
- Обратите внимание на тот факт, что адреса на схеме увеличиваются снизу вверх.



Виртуальное адресное пространство процесса  
(области нарисованы без соблюдения масштаба)

# Виртуальная память

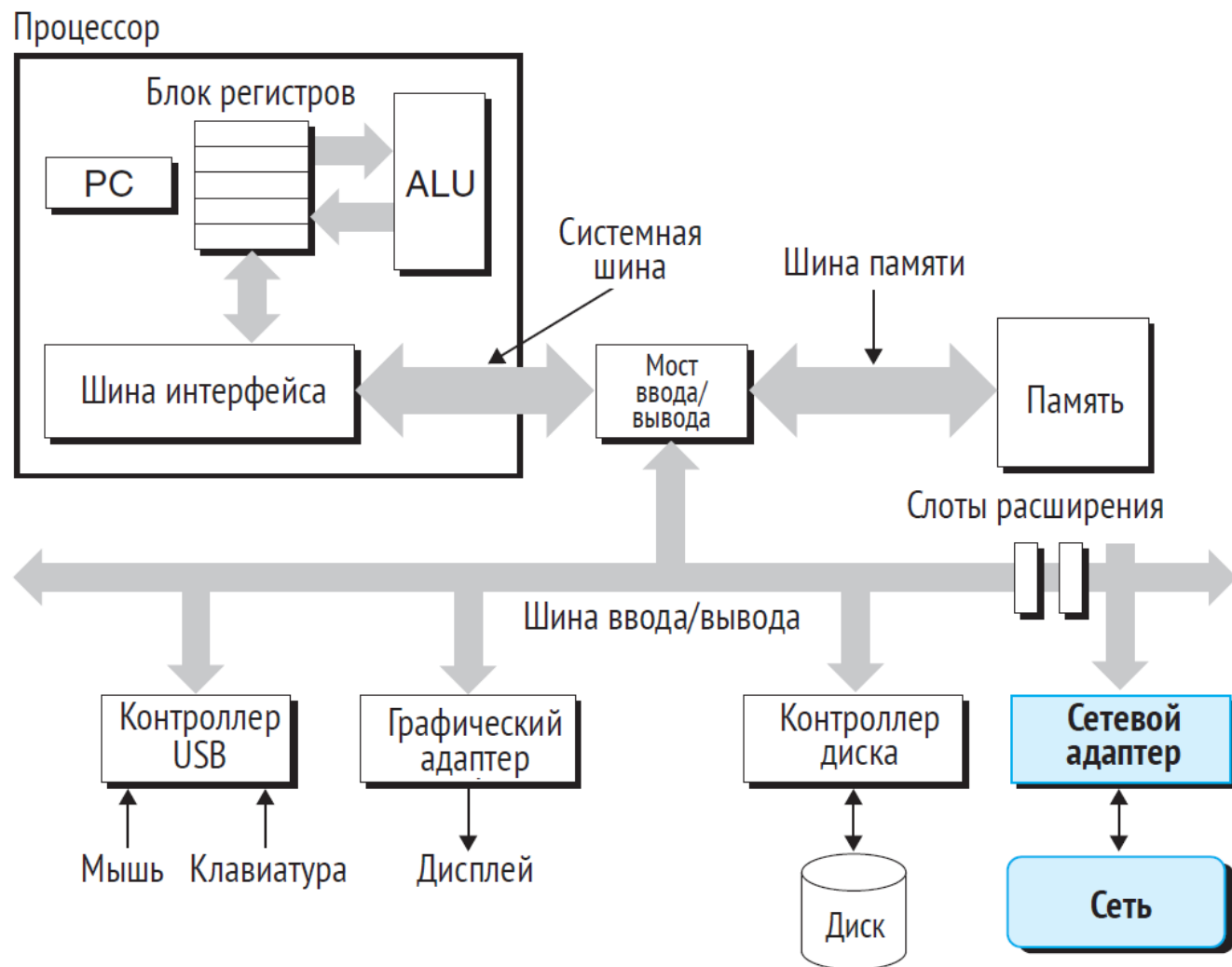
- Виртуальное адресное пространство, с точки зрения каждого процесса, состоит из некоторого числа четко определенных областей, каждая из которых выполняет свою функцию.
- Кратко рассмотрим каждую из них прямо сейчас, начав с меньших адресов и продвигаясь в сторону их увеличения.
  - **Программный код и данные.** Программный код каждого процесса всегда начинается с одного и того же адреса, за ним следуют ячейки памяти, соответствующие глобальным переменным. Область с кодом и данными инициализируется непосредственно содержимым выполняемого объектного файла, в нашем случае hello.
  - **Динамическая память (куча).** Непосредственно за кодом и данными следует область динамической памяти (кучи) программы. В отличие от областей с программным кодом и глобальными данными, размеры которых фиксируются, как только процесс начнет выполняться, динамическая память может расширяться и сокращаться в размерах во время выполнения программы, при вызове некоторых функций из стандартной библиотеки C, таких как malloc и free.
  - **Совместно используемые (разделяемые) библиотеки.** Ближе к середине адресного пространства находится область с программным кодом и данными совместно используемых библиотек, таких как стандартная библиотека C или библиотека математических функций. Понятие совместно используемой библиотеки является мощным, но в то же время несколько трудным для понимания.
  - **Стек.** В верхней части виртуального адресного пространства находится стек пользователя, который применяется компилятором для реализации вызовов функций. Как и динамическая память, стек пользователя может динамически расширяться и сокращаться в размерах во время выполнения программы. В частности, каждый раз, когда программа вызывает какую-либо функцию, размер стека увеличивается. Каждый раз, когда функция возвращает управление, стек сокращается.
  - **Виртуальная память ядра.** Верхняя часть адресного пространства зарезервирована для ядра. Прикладным программам запрещено читать содержимое этой области, записывать в нее данные или напрямую вызывать функции ядра.
- Для правильной работы виртуальной памяти требуется сложное взаимодействие аппаратных и программных средств операционной системы, включая аппаратное преобразование каждого адреса, генерируемого процессором. Основная идея заключается в том, чтобы сохранить содержимое виртуальной памяти процесса на диске, а затем использовать основную память как кеш диска.

# Файлы

- **Файл** – это всего лишь последовательность байтов, не более и не менее. Каждое устройство ввода/вывода, в том числе диски, клавиатуры, устройства отображения и даже сети, моделируется соответствующим файлом.
- Все операции ввода/вывода в системе выполняются путем чтения и записи в файлы посредством нескольких системных вызовов, образующих подсистему ввода/вывода Unix.
- Это простое и элегантное понятие файла, тем не менее, обладает глубоким смыслом, поскольку обеспечивает унифицированное представление всего разнообразия файлов, которые могут входить в состав системы.
- Например, прикладные программисты, манипулирующие содержимым дискового файла, могут быть абсолютно не знакомы с дисковыми технологиями и при этом чувствовать себя вполне комфортно. Более того, одна и та же программа будет прекрасно выполняться в разных системах, использующих разные дисковые технологии.

# Обмен данными в сетях

- До этого момента в нашем экскурсе мы рассматривали систему как изолированную совокупность аппаратных и программных средств.
- На практике современные системы часто соединены с другими системами посредством компьютерных сетей.
- С точки зрения отдельной системы, сеть можно рассматривать как еще одно устройство ввода/вывода, как показано на рисунке. →
- Когда система копирует некоторую последовательность байтов из основной памяти в сетевой адаптер, поток данных устремляется через сеть в другую машину, а не, скажем, в локальный дисковый накопитель.
- Аналогично система может читать данные, отправленные другими машинами, и сохранять в своей основной памяти.



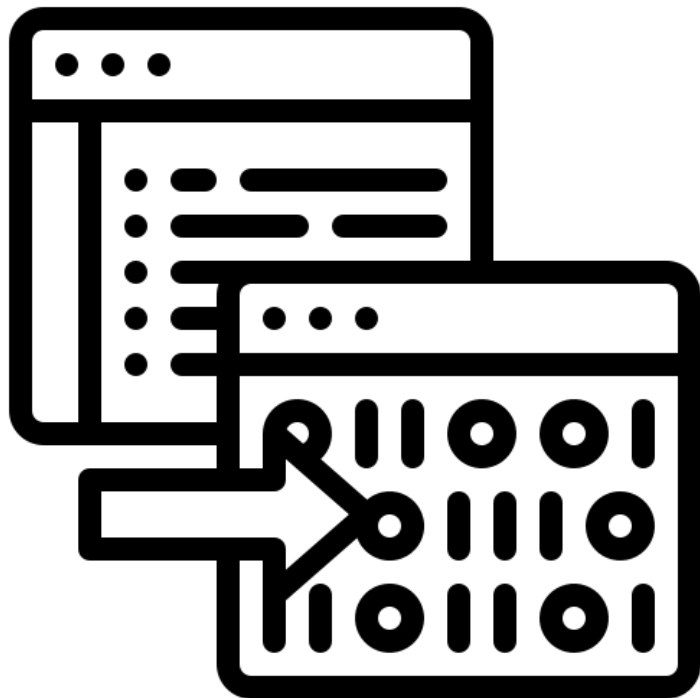
Сеть – это еще одно устройство ввода/вывода

# Продолжение примера работы программы hello



Дальнейший подробнейший разбор как выполняется программа на аппаратном и программном уровне, как все это взаимодействует и дает результат **описано** на 994 страницах **в книге:**

**«Рэндал Э. Брайант, Дэвид Р. О'Халларон  
Компьютерные системы: архитектура и  
программирование. 3-е изд. / пер. с англ. А.  
Н. Киселева. – М.: ДМК Пресс, 2022. – 994 с.»**



# Машинный код и язык ассемблера

# Таненбаум - Архитектура компьютера. 6-е изд, 2021.pdf

Белорусско-Российский университет, Кафедра «Программное обеспечение информационных технологий»



|                                                                         |     |
|-------------------------------------------------------------------------|-----|
| Приложение В. Программирование на языке ассемблера ...                  | 729 |
| Основные понятия .....                                                  | 730 |
| Язык ассемблера.....                                                    | 730 |
| Небольшая программа на языке ассемблера.....                            | 731 |
| Процессор 8088 .....                                                    | 732 |
| Цикл процессора .....                                                   | 732 |
| Регистры общего назначения .....                                        | 734 |
| Регистры-указатели .....                                                | 735 |
| Память и адресация .....                                                | 737 |
| Организация памяти и сегменты.....                                      | 737 |
| Адресация .....                                                         | 739 |
| Набор команд 8088 .....                                                 | 743 |
| Перемещение, копирование и арифметические команды.....                  | 745 |
| Логические операции, побитовые операции и операции сдвига .....         | 747 |
| Операции организации циклов и повторяющиеся<br>строковые операции ..... | 748 |
| Команды перехода и вызова .....                                         | 749 |
| Вызовы подпрограмм .....                                                | 751 |
| Системные вызовы и системные подпрограммы .....                         | 752 |
| Заключительные замечания о наборе команд .....                          | 755 |
| Ассемблер .....                                                         | 756 |
| Введение .....                                                          | 756 |
| Ассемблер as88 из набора АСК .....                                      | 757 |
| Некоторые отличия от других ассемблеров 8088 .....                      | 761 |
| Трассер .....                                                           | 762 |
| Команды трассера .....                                                  | 764 |
| Подготовительные действия .....                                         | 766 |
| Примеры .....                                                           | 767 |
| Hello World .....                                                       | 767 |



# Ассемблер

- **Ассемблер незаменим в таких вещах:**
  - драйверы;
  - программирование микроконтроллеров и встраиваемых процессоров;
  - куски операционных систем, где важно обеспечить скорость работы;
  - антивирусы (и вирусы).
- **Ассемблер** позволяет работать с процессором и памятью напрямую — и делать это очень быстро.

# Использование ассемблера

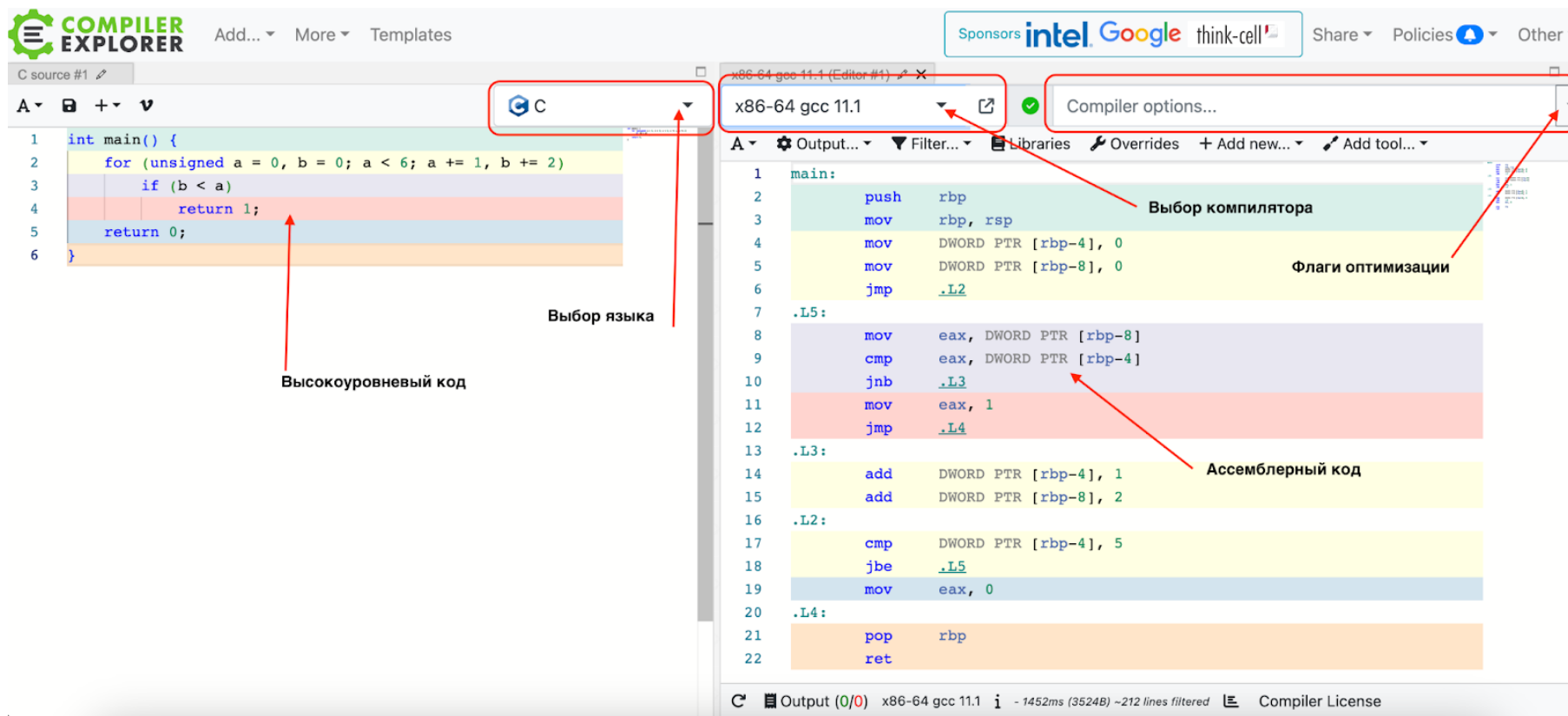
- **Повышение скорости реагирования участков программ,** написанных C++, например, или на других высокоуровневых языках. Это востребовано в игровых приставках с фиксированной производительностью. Ещё подходит для мультимедийных кодеков, для которых во главе угла – популярность, а не ресурсоемкость.
- **Разработка операционных систем.** Часто для написания ОС используется язык Си, он изначально и придумывался для создания одной из первых версий Unix. Ассемблер активно применяется для написания аппаратно зависимых участков кода, таких, например, как загрузчик ОС, HAL и ядро. В ядрах Windows или Linux тоже есть участки с ассемблерным кодом, но небольшие (для авторов на первом месте надёжность и переносимость). Иногда любительские ОС полностью пишутся на ассемблере. Пример – MenuetOS, который, кстати, уместается на дискете, а интерфейс представлен в виде многооконной графики.

# Использование ассемблера

- **Написание программ для микроконтроллеров (МК) и микропроцессоров.** Профессор Танненбаум, например, убеждён, что МК сейчас развиваются так же стремительно, как когда-то компьютеры нового времени. Язык ассемблер очень активно используется для программирования МК. И там частая операция – это перемещение отдельных байтов из одних ячеек памяти – в другие. Тот же профессор Танненбаум подчеркивает важность программирования МК, приводя в пример современные жилые дома, в которых где-то по 50 микроконтроллеров непременно имеется.
- **Разработка драйверов.** Ассемблер применяется для участков драйверов, связанных с аппаратным обеспечением. Вообще, главное требование к драйверам – это надёжность, поэтому их чаще пишут на высокоуровневых языках. Ведь в Windows NT и Linux, например, драйверы работают в режиме ядра, и любая ошибка может просто уничтожить систему.
- **Написание защитных программ, антивирусов и т.п.**
- **Разработка трансляторов для языков программирования.**

# Почему это сложно

- **Для того, чтобы писать программы на Ассемблере, нужно очень любить кремний:**
  - понимать архитектуру процессора;
  - знать устройство железа, которое работает с этим процессором;
  - знать все команды, которые относятся именно к этому типу процессоров;
  - уметь работать с данными в побайтовом режиме (забудьте о строках и массивах, ведь ваш максимум — это одна буква);
  - понимать, как в ограниченных условиях реализовать нужную функциональность.



**Compiler Explorer** — онлайн-инструмент, созданный Мэттом Годболтом для исследования компиляторов. В Compiler Explorer можно писать код на высокоуровневом языке и сразу видеть, какой ассемблерный код генерирует для него компилятор. А если это Python или Java, можно увидеть их байт-код.

Принцип работы инструмента простой: слева выбираем нужный язык, справа — версию компилятора. Затем в окно слева вводим код, а справа видим сгенерированный ассемблер.